

Practical 4

Group 44- DreamVille

Felicity Moyo - u24702791

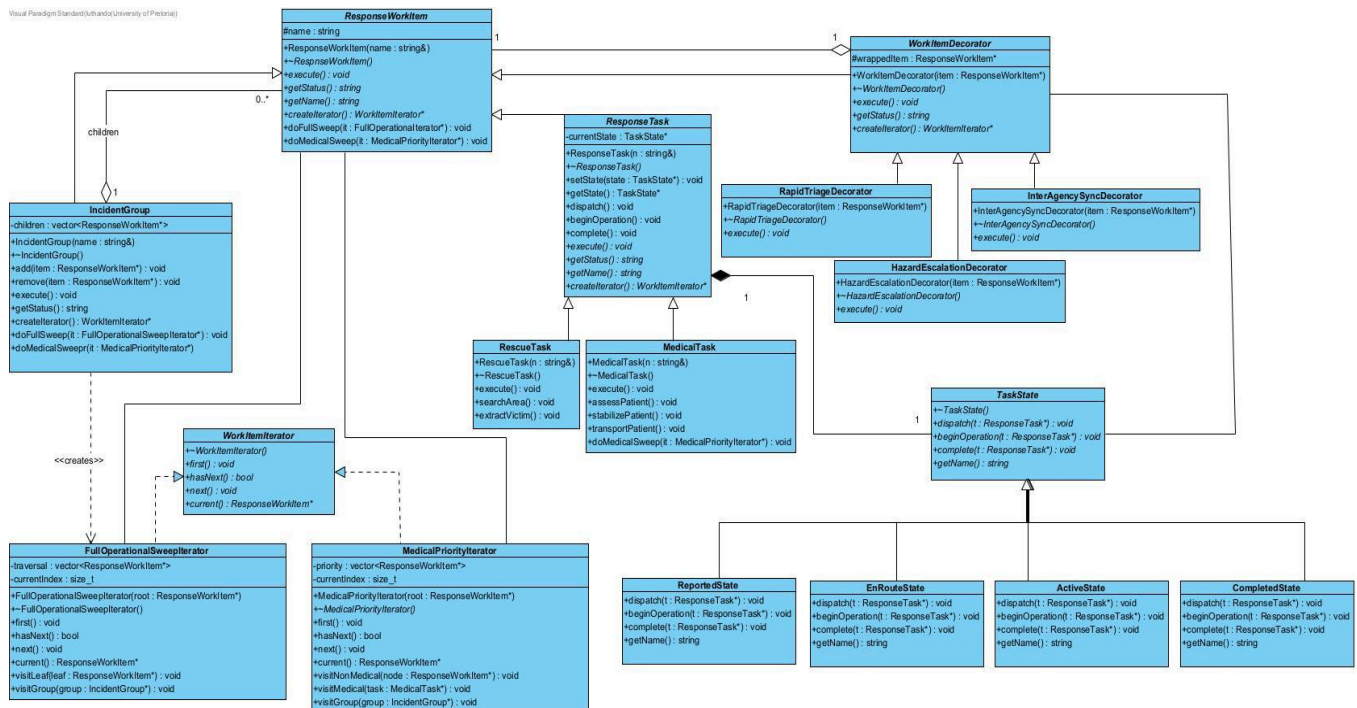
Tendai Jonasi - u25010230

Luthando Shilwaio - u25504747

Task 1

Chosen domain: Emergency Response

Our TaskForge is an emergency-response management system designed to coordinate rescue and medical operations during incidents such as wildfires or vehicle accidents. It involves multiple teams operating simultaneously across different sectors making it hard to reorganise the tasks and manage progress. This system solves this problem by providing structured representation of incident operations. Rescue and medical can be grouped and updated according to their states and made better with additional functionality such as rapid triage and hazard escalation.



Below are each of the design patterns and their participants:

1. Composite design pattern

Component-ResponseWorkItem

Composite-IncidentGroup

Leaf: MedicalTask and RescueTask

2. Decorator

Component-ResponseWorkItem
Decorator-WorkItemDecorator
ConcreteDecorator- RapidTriageDecorator, HazardEscalationDecorator,
InterAgencySyncDecorator

3.State

Context-ResponseTask
State-TaskState
ConcreteState- ReportedState, EnRouteState, ActiveState, CompletedState

4.Iterator

Aggregate-ResponseWorkItem
ConcreteAggregate-IncidentGroup
Iterator-WorkItemIterator
ConcreteIterator-FullOperationalSweepIterator, MedicalPriorityIterator

Task 2

Just implementations nothing written needed unless we have to mention anything

Task 3

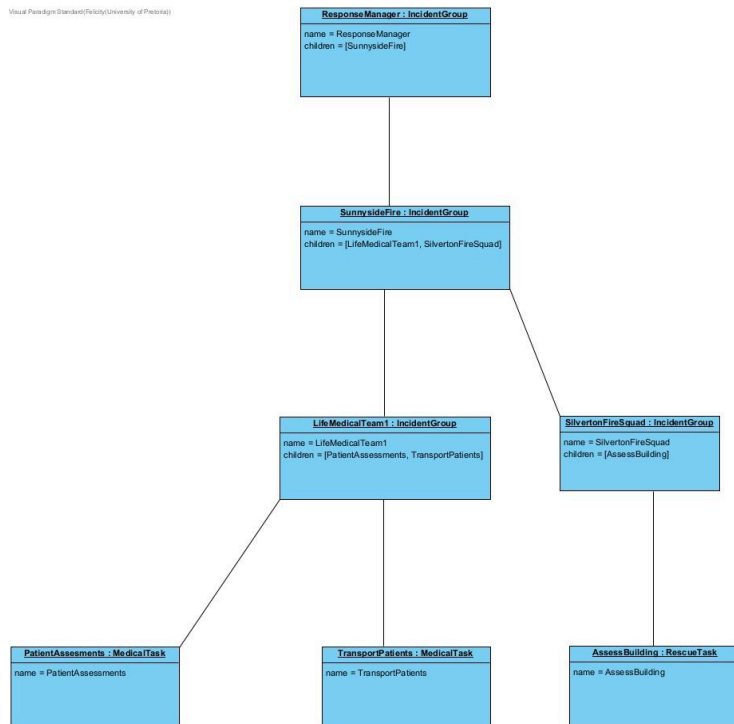
–Runtime Scenario : Adding extra responsibilities to Tasks

In emergency situations, basic tasks often require extra hands depending on how bad the incident is.

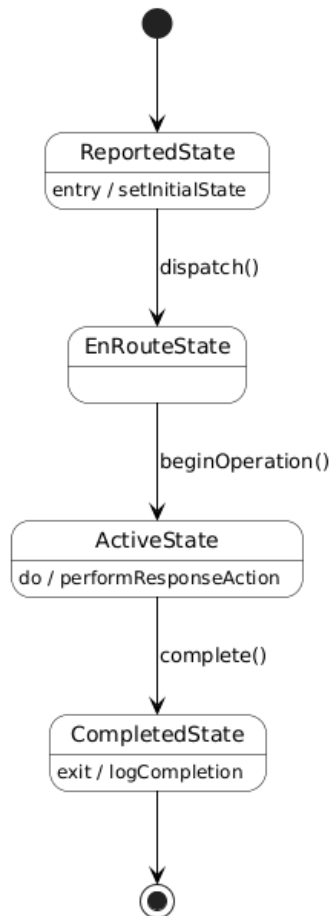
We create a standard medical task, eg treating a burn victim. Add priority, because the patient is crucial, we wrap the RapidTriage decorator to prioritize them. We then wrap that same task again in an Inter-Agency Sync decorator so other emergency services (like police or fire teams) get updated automatically.

Task 4

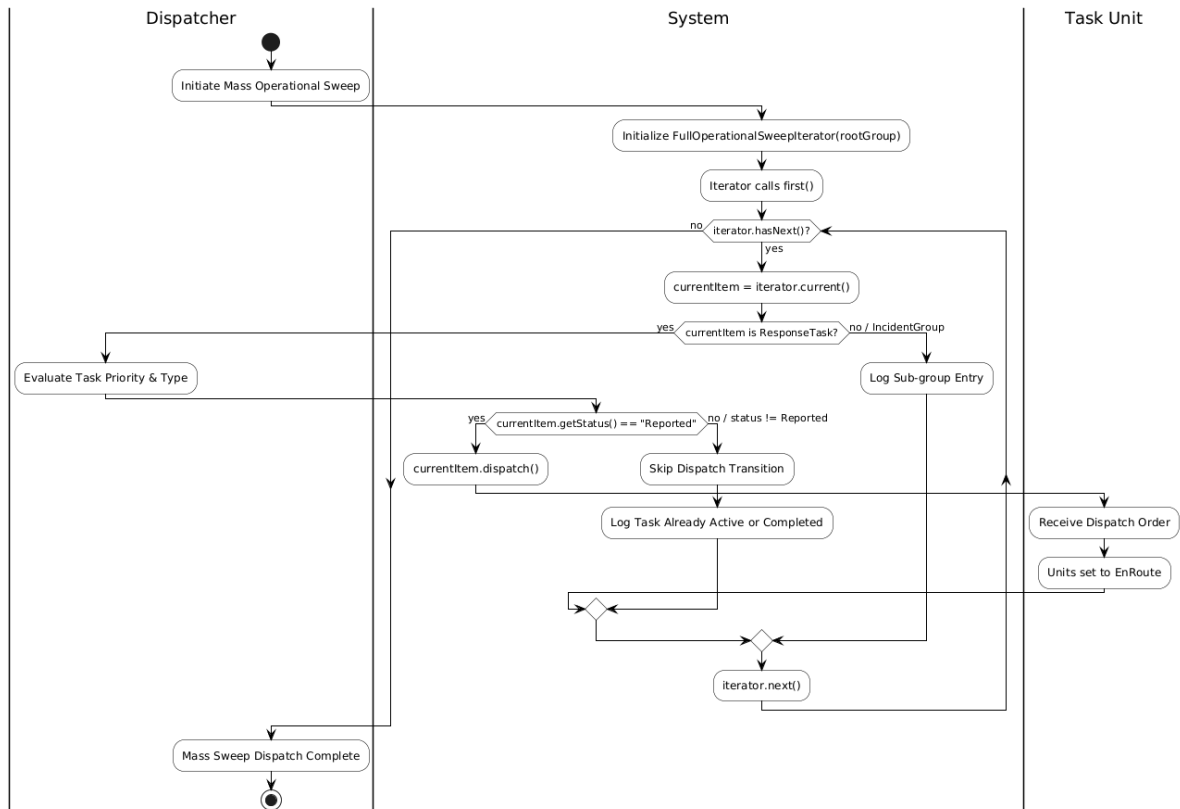
Object diagram



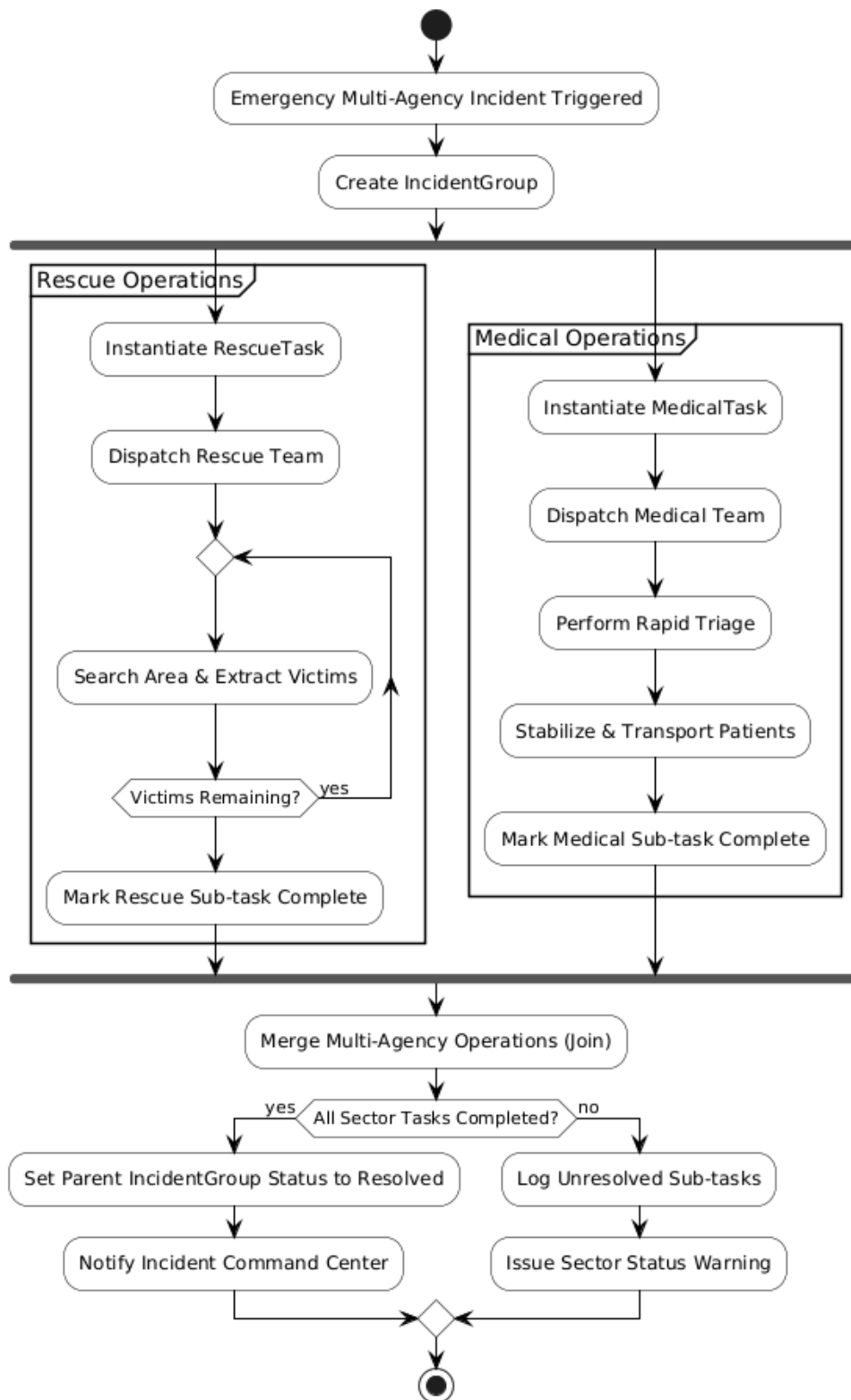
State diagram



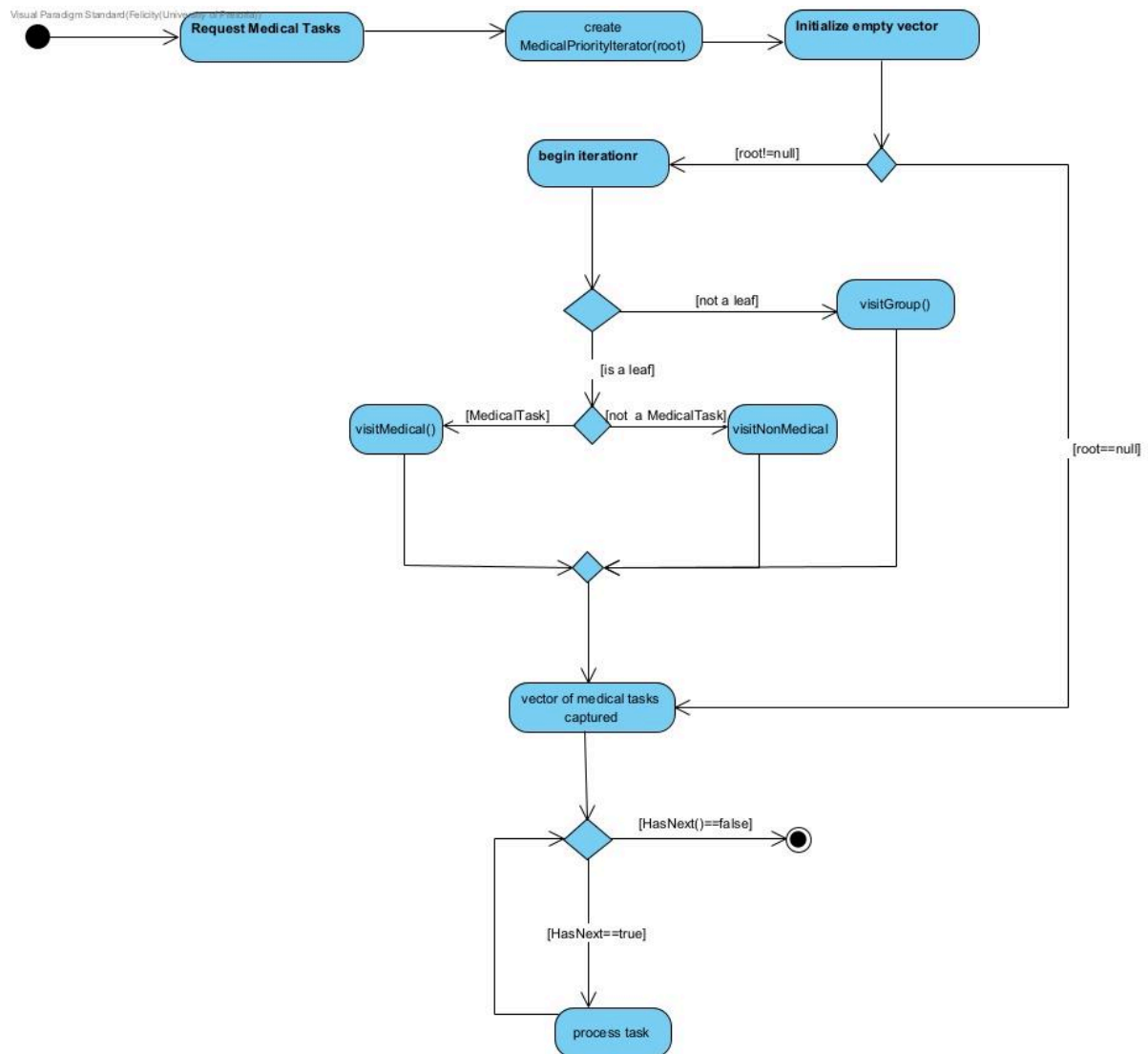
Activity diagram 1



Activity diagram 2



Activity Diagram 3



Task 5

Bug: Compilation failed during integration of the Composite pattern.

Cause: The implementation of *IncidentGroup::getName() const* didn't match any declaration in the header file.

Evidence:compiler output :

```

Problems 1 Output Debug Console Terminal Ports

CompletedState.cpp:9:51: warning: unused parameter 't' [-Wunused-parameter]
   9 | void CompletedState::beginOperation(ResponseTask* t) {
      |                                         ~~~~~^
CompletedState.cpp: In member function 'virtual void CompletedState::complete(ResponseTask*)':
CompletedState.cpp:13:45: warning: unused parameter 't' [-Wunused-parameter]
   13 | void CompletedState::complete(ResponseTask* t) {
      |                                         ~~~~~^
g++ -std=c++11 -g -Wall -Wextra -c IncidentGroup.cpp -o IncidentGroup.o
IncidentGroup.cpp:53:13: error: no declaration matches 'std::string IncidentGroup::getName() const'
   53 | std::string IncidentGroup::getName()const{
      |             ^~~~~~
IncidentGroup.cpp:53:13: note: no functions named 'std::string IncidentGroup::getName() const'
In file included from IncidentGroup.cpp:3:
IncidentGroup.h:8:7: note: 'class IncidentGroup' defined here
   8 | class IncidentGroup : public ResponseWorkItem {
      | ^~~~~~

```

Correction : Removed the redundant implementation from IncidentGroup.cpp and used the inherited version from ResponseWorkItem.

Verification; the code compiled successfully after correction.

Symptom: The compilation process failed during linking phases when running 'make', and produces the following error

"RescueTask.cpp:11: undefined reference to `ResponseTask::execute()'"

"MedicalTask.cpp:13: undefined reference to `ResponseTask::execute()'"

Cause: Derived leaf classes (RescueTask and MedicalTask) were attempting to invoke ResponseTask::execute(). However, `ResponseTask::execute()` was declared in `ResponseTask.h` without a corresponding implementation body in `ResponseTask.cpp`, leading to an unresolved symbol at link time

Evidence:

```

Undefined command: "quite". Try "help".
(gdb) quit
wkoz10@lulus:/mnt/c/Users/lutha/OneDrive/Desktop/COS214Pracs/Prac4$ ./taskforge
-bash: ./taskforge: No such file or directory
wkoz10@lulus:/mnt/c/Users/lutha/OneDrive/Desktop/COS214Pracs/Prac4$ make
g++ -std=c++11 -g -Wall -Wextra -O2 taskforge RapidInnageDecorator.o EnRouteState.o WorkItemDecorator.o ResponseTask.o ResponseWorkItem.o InterAgencySyncDecorator.o WorkItemIterator.o ReportedState.o ActiveState.o HazardEscalationDecorator.o RescueTask.o MedicalTask.o main.o CompletedState.o IncidentGroup.o /usr/bin/ld: RescueTask.o: in function `RescueTask::execute()':
/mnt/c/Users/lutha/OneDrive/Desktop/COS214Pracs/Prac4/RescueTask.cpp:11: undefined reference to `ResponseTask::execute()'
/usr/bin/ld: MedicalTask.o: in function `MedicalTask::execute()':
/mnt/c/Users/lutha/OneDrive/Desktop/COS214Pracs/Prac4/MedicalTask.cpp:13: undefined reference to `ResponseTask::execute()'
collect2: error: ld returned 1 exit status
make: *** [Makefile:8: taskforge] Error 1
wkoz10@lulus:/mnt/c/Users/lutha/OneDrive/Desktop/COS214Pracs/Prac4$

```

Correction: Added the missing function body `void ResponseTask::execute() {}` inside `ResponseTask.cpp`. Re-ran `make clean` & `make`, which produced the `taskforge` executable without any build or linking errors.

Docker gdb:

Executable:taskforge

The base image was successfully constructed containing all specified compiler and debugging tooling (g++, make, gdb, valgrind). The program target taskforge builds cleanly inside /app

Breakpoint Verification: Setting break ResponseTask::dispatch successfully paused execution right before invoking state-dependent behavior.

```
[Treat Rescued Resident - Cabin 12: Reported] ]
whozlul@lulus:/mnt/c/Users/lutha/OneDrive/Desktop/COS214Pracs/Prac4$ docker run -it --rm taskforge bash
root@ec97e054ef0e:/app# gdb ./taskforge
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

--Type <RET> for more, q to quit, c to continue without paging--
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforge...
(gdb) break ResponsTask::dispatch
Function "ResponsTask::dispatch" not defined.
Make breakpoint pending on future shared library load? (y or [n]) n
(gdb) break ResponseTask::dispatch
Breakpoint 1 at 0x6d22: file ResponseTask.cpp, line 23.
(gdb) run
Starting program: /app/taskforge
warning: Error disabling address space randomization: Operation not permitted
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
=====
COMPOSITE DEMO
```

```
=====
Task: Extract Resident - Cabin 12
Current state: Reported

[Attempting beginOperation() too early]
[Invalid Action] Cannot begin operation while task is still in ReportedState.

[Executing valid lifecycle sequence]

Breakpoint 1, ResponseTask::dispatch (this=0x55d796ac10f0) at ResponseTask.cpp:23
23     if(currentState){
(gdb) print currentState
No symbol "currentState" in current context.
(gdb) print currentState
$1 = (TaskState *) 0x55d796ac1160
(gdb) step
24         currentState->dispatch(this);
(gdb) print currentState
$2 = (TaskState *) 0x55d796ac1160
(gdb) quit
A debugging session is active.

        Inferior 1 [process 13] will be killed.

Quit anyway? (y or n) y
root@ec97e054ef0e:/app# quit
bash: quit: command not found
root@ec97e054ef0e:/app# exit
exit
```

Valgrind


```
Demo complete.
==24582==
==24582== HEAP SUMMARY:
==24582==    in use at exit: 0 bytes in 0 blocks
==24582== total heap usage: 46 allocs, 46 frees, 74,781 bytes allocated
==24582==
==24582== All heap blocks were freed -- no leaks are possible
==24582==
==24582== For lists of detected and suppressed errors, rerun with: -s
==24582== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Task 6

On the 4th of September 2026 the group held a meeting where the overall design of Task Forge was decided. We discussed which objects would exist and how they would interact with each other. From there Luthando was tasked with designing the UML diagram, one activity diagram, implementing the decorator logic and creating the Docker file. Tendai was assigned the state diagram, one activity diagram and implementing the state behaviour. Lastly Felicity worked on the object diagram, the remaining activity diagram, the composite and iterator implementations. To share code and designs, the team worked on a private Github repository. Each member worked on a separate branch where they handled their assigned implementations using commits. For example when partially completed files were initially committed, commit messages such as “draft decorator” were used and once fully functional they were committed as “fully working implementation...”. This helped each member track each other's progress. In the cases where patterns overlapped pull requests were made and code was corrected and fixed to avoid merge conflicts. For example as the ResponseTask class is shared between multiple patterns multiple members worked on it in their own branch, the Pull request “Getting code for state” was used to combine the composite and state logic. Once the design was fully functional it was merged to the main branch. The team also used a dedicated document branch where diagrams were uploaded, shared and changed as the design itself changed. Finally each member worked on the pdf document. In conclusion the team worked seamlessly and collaboratively with good communication and task delegation leading to the successful completion of Task Forge.